МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Дисциплина:
Операционные системы и архитектура компьютера

Тема:
Управление процессами в ОС. Обеспечение обмена данными между процессами посредством каналов

Выполнил:
Студент группы М8О-203БВ-25
К. П. Смаге

Преподаватель:
Е. С. Миронов

Москва, 2026 г.

1 Цель работы

Приобретение практических навыков в:

- управлении процессами в операционных системах семейств Linux (POSIX) и Windows (Win32 API);
- обеспечении межпроцессного взаимодействия (IPC) посредством неименованных каналов (`pipe`).

2 Краткое описание архитектуры решения

Разработанное решение реализует требования варианта 20. Программный комплекс состоит из двух отдельных исполняемых файлов: управляющей родительской программы (`parent`) и рабочей дочерней программы (`child`).

Алгоритм функционирования системы включает следующие этапы:

1. Родительский процесс считывает со стандартного потока ввода две строки, содержащие пути к выходным файлам для первого и второго дочерних процессов.
2. Родительский процесс инициализирует два экземпляра класса `Process` (`child1` и `child2`). Для каждого дочернего процесса создается однонаправленный неименованный канал (`pipe1` и `pipe2`).
3. Стандартный поток ввода (`stdin`) каждого дочернего процесса перенаправляется на читающий конец соответствующего канала, а стандартный поток вывода (`stdout`) перенаправляется в целевой выходной файл на диске. Стандартный поток ошибок (`stderr`) остается привязанным к терминалу для вывода диагностических сообщений и идентификаторов процессов (PID).
4. Родительский процесс построчно считывает пользовательский ввод и осуществляет фильтрацию согласно условию варианта 20:
   - если длина строки превышает 10 символов (length > 10), строка пересылается через `pipe2` процессу `child2`;
   - если длина строки не превышает 10 символов (length ≤ 10), строка пересылается через `pipe1` процессу `child1`.
5. Каждый дочерний процесс в цикле считывает поступающие строки из своего стандартного ввода, инвертирует порядок символов в строке и записывает результат в стандартный поток вывода (целевой файл).
6. При получении сигнала конца ввода (EOF) родительский процесс закрывает пишущие дескрипторы обоих каналов, сигнализируя дочерним процессам о завершении потока данных, и ожидает их штатного завершения, предотвращая образование процессов-зомби.

Для достижения кроссплатформенности применен паттерн изоляции платформенного слоя: бизнес-логика в `src/parent.cpp` и `src/child.cpp` не содержит прямых системных вызовов ОС и взаимодействует с ресурсами ядра исключительно через абстракцию `Process` (`include/process.hpp`), реализующую идиому RAII. Выбор конкретной реализации (`src/posix/process_posix.cpp` для Linux или `src/win/process_win.cpp` для Windows) выполняется системой сборки CMake на этапе конфигурации.

В таблице 1 приведено сопоставление системных примитивов, используемых в платформенных модулях для Linux и Windows.

Операция ОС	Linux (POSIX API)	Windows (Win32 API)
Создание канала	`pipe(pipefd)`	`CreatePipe(&pipefd[0], &pipefd[1], &sa, 0)`
Открытие файла вывода	`open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644)`	`CreateFileA(path, GENERIC_WRITE, ..., CREATE_ALWAYS, ...)`
Перенаправление потоков	`dup2(fd, STDIN_FILENO / STDOUT_FILENO)`	Поля `hStdInput`, `hStdOutput` в `STARTUPINFOA`
Изоляция конца записи	`close(pipefd[1])` в дочернем процессе	`SetHandleInformation(pipefd[1], HANDLE_FLAG_INHERIT, 0)`
Порождение процесса	`fork()` + `execl(path, path, nullptr)`	`CreateProcessA(nullptr, cmd.data(), ..., &si, &pi)`
Запись в канал	`write(write_fd, buf, size)`	`WriteFile(write_fd, buf, size, &written, nullptr)`
Ожидание завершения	`waitpid(pid, &status, 0)`	`WaitForSingleObject(pid, INFINITE)`
Освобождение ресурсов	`close(fd)`	`CloseHandle(handle)`

Таблица 1: Сопоставление системных вызовов в реализациях для Linux и Windows

3 Листинг ключевых фрагментов кода

3.1 Кроссплатформенный интерфейс управления процессом

Заголовочный файл `include/process.hpp` инкапсулирует платформенные типы дескрипторов (`int` и `pid_t` в POSIX, `HANDLE` в Win32 API) и предоставляет единый RAII-интерфейс для запуска дочернего процесса, записи строк в канал и синхронизации завершения.

```
#ifdef _WIN32
    #include <windows.h>
    using pipe_handle_t = HANDLE;
    using proc_handle_t = HANDLE;
#else
    #include <unistd.h>
    #include <sys/types.h>
    using pipe_handle_t = int;
    using proc_handle_t = pid_t;
#endif

class Process {
public:
    Process(
        const std::string& binary_path,
        const std::string& output_file_path
    );

    ~Process();

    Process(const Process&) = delete;
    Process& operator=(const Process&) = delete;

    void send_line(const std::string& line);
    void close_input();
    void wait();

    unsigned long get_pid() const;

private:
    pipe_handle_t write_fd;
    proc_handle_t pid;
```

```
    bool is_closed;
};
```

Листинг 1 — Объявление класса `Process` (`include/process.hpp`)

3.2 Маршрутизация данных в родительском процессе

Функция `parent()` в файле `src/parent.cpp` отвечает за чтение имен файлов, создание двух дочерних процессов, распределение входных строк по критерию длины (≤ 10 или > 10 символов) и корректное закрытие каналов.

```
void parent() {
    std::cout << "Enter output file for Child1: ";
    std::string file1{};
    if (!std::getline(std::cin, file1) || file1.empty()) {
        throw std::runtime_error("Failed to read filename");
    }

    std::cout << "Enter output file for Child2: ";
    std::string file2{};
    if (!std::getline(std::cin, file2) || file2.empty()) {
        throw std::runtime_error("Failed to read filename");
    }

    Process child1(CHILD_PATH, file1);
    Process child2(CHILD_PATH, file2);

    std::string line{};

    std::cout << "Enter lines for processing (Ctrl+D / Ctrl+C to finish): \n";
    while (std::getline(std::cin, line)) {
        if (line.length() > 10) {
            child2.send_line(line);
        } else {
            child1.send_line(line);
        }
    }

    child1.close_input();
    child2.close_input();

    child1.wait();
    child2.wait();
}
```

Листинг 2 — Логика диспетчеризации строк в родительском процессе (`src/parent.cpp`)

3.3 Реализация IPC и порождения процесса в ОС Linux (POSIX)

В файле `src/posix/process_posix.cpp` создание процесса выполняется связкой вызовов `pipe()`, `fork()`, `open()`, `dup2()` и `execl()`. В каждом системном вызове проверяется код возврата с выводом ошибки через `perror`.

```
Process::Process(
        const std::string& binary_path,
        const std::string& output_file_path
    ) {
        int file_fd = open(
            output_file_path.c_str(),
            O_WRONLY | O_CREAT | O_TRUNC,
            0644
        );

        if (file_fd == -1) {
            perror("open");
            throw std::runtime_error("Failed to open output file: " + output_file_path);
```

```
        }

        int pipefd[2]{};

        if (pipe(pipefd) == -1) {
            perror("pipe");
            close(file_fd);

            throw std::runtime_error("Failed to create pipe");
        }

        pid_t fork_rc = fork();

        if (fork_rc == -1) {
            // failed
            perror("fork");

            close(pipefd[0]);
            close(pipefd[1]);

            throw std::runtime_error("Failed to fork");
        } else if (fork_rc == 0) {
            // child
            if (dup2(pipefd[0], STDIN_FILENO) == -1) {
                perror("dup2 stdin");
                _exit(EXIT_FAILURE);
            }

            if (dup2(file_fd, STDOUT_FILENO) == -1) {
                perror("dup2 stdout");
                _exit(EXIT_FAILURE);
            }

            close(file_fd);
            close(pipefd[0]);
            close(pipefd[1]);

            execl(
                binary_path.c_str(),
                binary_path.c_str(),
                static_cast<char*>(nullptr)
            );

            perror("execl");
            _exit(127);
        } else {
            // parent
            this->pid = fork_rc;
            this->write_fd = pipefd[1];
            this->is_closed = false;

            close(pipefd[0]);
            close(file_fd);
        }
}

void Process::send_line(const std::string& line) {
    if (is_closed || write_fd == -1) {
        throw std::runtime_error("Attempted to write to a closed or invalid pipe");
    }

    std::string f_line = line;

    if (f_line.empty() || f_line.back() != '\n') {
        f_line += '\n';
    }

    ssize_t bytes_written = write(write_fd, f_line.data(), f_line.size());

    if (bytes_written == -1) {
        perror("write");
        throw std::runtime_error("Failed to write data to pipe");
    }
```

```
}
```

Листинг 3 — Порождение процесса и перенаправление дескрипторов в POSIX (`src/posix/process_posix.cpp`)

3.4 Реализация IPC и порождения процесса в ОС Windows (Win32 API)

В файле `src/win/process_win.cpp` настройка наследуемости дескрипторов (`SECURITY_ATTRIBUTES`), изоляция пишущего конца трубы (`SetHandleInformation`) и привязка стандартных потоков (`STARTUPINFOA`) выполняются в родительском процессе перед вызовом `CreateProcessA`.

```
Process::Process(
    const std::string& binary_path,
    const std::string& output_file_path
) {
    SECURITY_ATTRIBUTES sa{};
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.bInheritHandle = TRUE;
    sa.lpSecurityDescriptor = nullptr;

    HANDLE file_fd = CreateFileA(
        output_file_path.c_str(),
        GENERIC_WRITE,
        FILE_SHARE_READ,
        &sa,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        nullptr
    );
    if (file_fd == INVALID_HANDLE_VALUE) {
        std::cerr << "CreateFileA error: " << GetLastError() << '\n';
        throw std::runtime_error("Failed to open output file: " + output_file_path);
    }

    HANDLE pipefd[2]{INVALID_HANDLE_VALUE, INVALID_HANDLE_VALUE};
    if (!CreatePipe(&pipefd[0], &pipefd[1], &sa, 0)) {
        std::cerr << "CreatePipe error: " << GetLastError() << '\n';
        CloseHandle(file_fd);
        throw std::runtime_error("Failed to create pipe");
    }

    if (!SetHandleInformation(pipefd[1], HANDLE_FLAG_INHERIT, 0)) {
        std::cerr << "SetHandleInformation error: " << GetLastError() << '\n';
        CloseHandle(file_fd);
        CloseHandle(pipefd[0]);
        CloseHandle(pipefd[1]);
        throw std::runtime_error("Failed to set handle information");
    }

    STARTUPINFOA si{};
    si.cb = sizeof(STARTUPINFOA);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = pipefd[0];
    si.hStdOutput = file_fd;
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);

    PROCESS_INFORMATION pi{};
    std::string cmd = binary_path;

    BOOL create_rc = CreateProcessA(
        nullptr,
        cmd.data(),
        nullptr,
        nullptr,
        TRUE,
        0,
        nullptr,
        nullptr,
```

```
        &si,
        &pi
    );

    if (!create_rc) {
        std::cerr << "CreateProcessA error: " << GetLastError() << '\n';
        CloseHandle(file_fd);
        CloseHandle(pipefd[0]);
        CloseHandle(pipefd[1]);
        throw std::runtime_error("Failed to create process");
    }

    this->pid = pi.hProcess;
    this->write_fd = pipefd[1];
    this->is_closed = false;

    CloseHandle(pi.hThread);
    CloseHandle(pipefd[0]);
    CloseHandle(file_fd);
}

void Process::send_line(const std::string& line) {
    if (is_closed || write_fd == INVALID_HANDLE_VALUE) {
        throw std::runtime_error("Attempted to write to a closed or invalid pipe");
    }

    std::string f_line = line;
    if (f_line.empty() || f_line.back() != '\n') {
        f_line += '\n';
    }

    DWORD bytes_written = 0;
    BOOL write_rc = WriteFile(
        write_fd,
        f_line.data(),
        static_cast<DWORD>(f_line.size()),
        &bytes_written,
        nullptr
    );

    if (!write_rc) {
        std::cerr << "WriteFile error: " << GetLastError() << '\n';
        throw std::runtime_error("Failed to write data to pipe");
    }
}
```

Листинг 4 — Порождение процесса и настройка каналов в Windows (`src/win/process_win.cpp`)

3.5 Обработка строк в дочернем процессе

Дочерний процесс (`src/child.cpp`) считывает строки из `std::cin`, инвертирует их функцией `reverse_string` и записывает результат в перенаправленный поток `std::cout`.

```
void reverse_string(std::string& line) {
    std::reverse(line.begin(), line.end());
}

void child() {
    std::string line{};

    while (std::getline(std::cin, line)) {
        reverse_string(line);
        std::cout << line << std::endl;
    }
}
```

Листинг 5 — Инвертирование строк и вывод телеметрии в `src/child.cpp`

4 Результаты работы

4.1 Консольный вывод и логи выполнения

Ниже приведен фактический протокол работы программы в терминале с отображением идентификаторов процессов (PID), подтверждающий параллельное существование родительского и двух дочерних процессов, корректную маршрутизацию строк по длине и штатное завершение по сигналу EOF:

```
$ ./parent
[Parent] PID: 463
Enter output file for Child1: short_lines.txt
Enter output file for Child2: long_lines.txt
[Parent] Created Child1 (PID: 479)
[Parent] Created Child2 (PID: 480)
Enter lines for processing (Ctrl+D / Ctrl+C to finish):
12345
1234567
12345678910
123456789101112
12
123456789876543210
[Parent] Closed Child1
[Parent] Closed Child2

$ cat short_lines.txt
54321
7654321
21

$ cat long_lines.txt
01987654321
211101987654321
012345678987654321
```

5 Выводы

В результате выполнения лабораторной работы реализован кроссплатформенный программный комплекс межпроцессного взаимодействия, осуществляющий потоковую фильтрацию и инвертирование строк с использованием каналов. В ходе разработки были учтены архитектурные различия между моделями управления процессами в рассматриваемых ОС.

В процессе работы мною были изучены и отработаны навыки перенаправления потоков ввода/вывода между процессами и практика безопасного управления ресурсами.